

计算机设计实践

最终实验报告

单周期 CPU 与五级流水线 CPU 扩展指令验收

姓名	黄宇轩
学号	2024302181173
实验内容	RISC-V 单周期 CPU 与流水线 CPU 指令扩展、仿真验收与上板展示
报告日期	2026 年 6 月 20 日
GitHub	https://github.com/yuxuanhuang641-crypto/computer-design-practice-final-2024302181173

一、实验任务与完成情况

最终验收要求在原有 RISC-V 单周期 CPU 和五级流水线 CPU demo 基础上补充扩展指令，并能够通过 iverilog 测试、学号排序程序和开发板展示。本次修改围绕译码、ALU 运算、立即数扩展、下一条 PC 计算、流水线转发和控制冲刷展开。

完成情况：

单周期 CPU：完成 slt/sltu、andi/ori/xori、srli/srai/slli、slti/sltiu、bne/bge/bgeu/blt/bltu、jalr。实现重点是 ctrl 译码、EXT 立即数扩展、ALU 运算/比较和 NPC 跳转目标计算。

流水线 CPU：在单周期扩展基础上增加 beq、jal、jalr 的流水线支持，并通过段寄存器保存控制信号、通过转发和 flush 处理冒险。

自动评分脚本结果如下，所有小测试点均通过，得分 100/100。

图 1 自动测试结果：Score 100/100

```
PS E:\computerlab\final_acceptance> python tools\run_score_all.py --no-compile
[PASS] sc_slt_signed: 4/4
[PASS] sc_sltu_unsigned: 4/4
[PASS] sc_logic_immediate: 6/6
[PASS] sc_shift_immediate: 5/5
[PASS] sc_slti_sltiu: 7/7
[PASS] sc_branch_group: 10/10
[PASS] sc_jalr_link_jump: 6/6
[PASS] pl_slt_signed: 4/4
[PASS] pl_sltu_unsigned: 4/4
[PASS] pl_logic_immediate: 6/6
[PASS] pl_shift_immediate: 5/5
[PASS] pl_slti_sltiu: 7/7
[PASS] pl_branch_group: 10/10
[PASS] pl_beq_flush: 2/2
[PASS] pl_jal_flush: 6/6
[PASS] pl_jalr_flush: 6/6
Score: 100/100
Raw score: 92/92
```

二、总体实现原理

2.1 单周期 CPU 数据通路

单周期 CPU 的核心特点是一条指令在一个时钟周期内走完整条数据通路：PC 取指，ctrl 根据 opcode/funct3/funct7 产生控制信号，RF 读出 rs1/rs2，EXT 产生立即数，ALU 完成运算或分支比较，dm 在 lw/sw 时访存，最后由 WDSel 选择 ALU、内存或 PC+4 写回寄存器。

单周期修改重点：补充指令译码、扩充 ALUOp、增加 jalr 的 RS1 输入和跳转目标计算。因为单周期没有阶段重叠，所以不需要转发和阻塞，控制信号在当前周期内直接使用。

2.2 五级流水线 CPU 数据通路

流水线 CPU 将一条指令拆成 IF、ID、EX、MEM、WB 五个阶段。每个阶段只做一部分工作，阶段之间用 IF/ID、ID/EX、EX/MEM、MEM/WB 流水寄存器保存数据和控制信号。这样多个指令可以重叠执行，但也会出现数据冒险和控制冒险。

流水线额外段寄存器：IF/ID 保存 PC 和指令；ID/EX 保存译码结果、立即数、寄存器读数和控制信号；EX/MEM 保存 ALU 结果、写内存数据和访存控制；MEM/WB 保存内存读数、ALU 结果、rd 和写回选择。控制信号必须随指令逐级传递，否则后级无法知道这条指令是否要访存、写回或写内存。

三、问题回答

本节把老师可能追问的运算指令、跳转/分支指令、单周期与流水线区别、冒险处理以及 im/RF/dm/ctrl 等模块统一整理。回答时可以先说指令类型和数据通路，再指出代码位置。

3.1 运算指令：add、addi、slt、slli

add x3, x1, x2: 这是 R-type 指令。CPU 从 instr 中取出 rs1=x1、rs2=x2、rd=x3；RF 读出 x1 和 x2；ctrl 看到 opcode=0110011 且 funct3/funct7 对应 add，就让 ALUOp=add、ALUSrc=0、RegWrite=1、WDSel=ALU；ALU 做 A+B，结果在写回阶段写入 x3，PC 正常变成 PC+4。

addi x5, x0, 0xff: 这是 I-type 算术立即数指令。rs1 是 x0，rd 是 x5，立即数来自 instr[31:20] 并由 EXT 做符号扩展；ctrl 设置 ALUSrc=1，所以 ALU 的第二个输入来自 imm；ALU 做 x0+imm，最后写回 x5。

slt x27, x25, x26: 这是 R-type 有符号比较指令。RF 读出 x25、x26 后，ALU 按有符号大小比较 $A < B$ ；成立输出 1，不成立输出 0，再通过 WDSel=ALU 写回 x27。sltu 与它类似，但比较时按无符号数处理。

slli x27, x19, 16: 这是 I-type 移位指令。RF 读出 x19，立即数字段给出 shamt=16；ALU 执行左移，移位量只取低 5 位，因为 RV32I 一次最多移动 0 到 31 位；结果写回 x27。

```
ctrl.v: opcode/funct3/funct7 -> ALUOp
alu.v : ALUOp_add   -> C = A + B
alu.v : ALUOp_slt   -> C = (A < B) ? 32'd1 : 32'd0
alu.v : ALUOp_sll   -> C = A << B[4:0]
```

3.2 跳转和分支指令：beq、bne、jal、jalr

beq x7, x23, label: 这是 B-type 条件分支。RF 读出 x7 和 x23，ALU 判断二者是否相等；如果相等，ctrl 让 NPCOp 选择分支，NPC=当前 PC+B 型立即数；如果不相等，NPC=PC+4。beq 不写寄存器，也不写数据内存。

bne x5, x7, label: bne 的流程和 beq 基本一样，只是条件相反：ALU 判断 $x5 \neq x7$ 。条件成立时 NPC 选择 PC+branch_imm，条件不成立时顺序执行 PC+4。

jal x1, target: jal 是 J-type 无条件跳转，并保存返回地址。它同时做两件事：x1=当前 PC+4，PC=当前 PC+J 型立即数。因此 ctrl 需要设置 RegWrite=1、WDSel=PC+4、NPCOp=jump。

jalr x0, x1, 0: jalr 是寄存器间接跳转，目标地址是 $(rs1 + imm) \& 0xffffffe$ 。对于 jalr x0,x1,0，就是跳到 x1 保存的返回地址；rd 是 x0，所以不保存新的返回地址，但跳转仍然发生。代码中 NPC.v 增加 RS1 输入，就是为了让 NPC 能算出这个目标地址。

```
NPC_BRANCH: NPC = PC + IMM
NPC_JUMP   : NPC = PC + IMM
NPC_JALR   : NPC = (RS1 + IMM) & 32'hffff_fffe
```

3.3 单周期 CPU 与流水线 CPU 的区别

单周期 CPU: 一条指令在一个时钟周期内完成取指、译码、读寄存器、ALU、访存、写回和下一条 PC 计算。结构直观，但时钟周期必须足够长，能容纳最慢指令的完整路径。

五级流水线 CPU: 把指令拆成 IF、ID、EX、MEM、WB 五级。IF 取指，ID 译码和读寄存器，EX 做 ALU 或分支判断，MEM 访问数据内存，WB 写回寄存器。多个指令重叠执行，所以吞吐率更高。

额外段寄存器: 流水线需要 IF/ID、ID/EX、EX/MEM、MEM/WB。IF/ID 保存 PC 和 instr；ID/EX 保存 rd、rs1、rs2、RD1、RD2、imm 和控制信号；EX/MEM 保存 ALU 结果、写内存数据和访存控制；MEM/WB 保存内存读数、ALU 结果、rd、RegWrite 和 WDSel。

为什么要保存控制信号: 因为指令不是一个周期完成。例如 lw 在 ID 阶段已经知道要 MemRead、RegWrite、WDSel=MEM，但真正读内存存在 MEM 阶段，真正写回在 WB 阶段，所以控制信号必须跟着这条指令逐级传递。

3.4 流水线冒险：RAW、load-use、转发和冲刷

RAW 数据冒险: 后一条指令要读的寄存器，正好是前一条指令还没有写回的 rd。例如 add 产生 x3，紧跟着 sub 使用 x3，此时 sub 在 EX 阶段需要的值可能还没有回到 RF。

数据转发：PLCPU 中用 mem_forward_value、wb_forward_value、forward_RD1、forward_RD2 判断 EX 阶段 rs1/rs2 是否等于 MEM/WB 阶段即将写回的 rd。如果相等，就直接把结果送回 ALU 输入端，不等寄存器堆写回。

load-use 冒险：典型情况是 lw 后面立即使用同一个 rd。因为 lw 的数据到 MEM 阶段才从 dm 出来，经典解决方法是检测 EX_MemRead 且 EX_rd 等于 ID 阶段源寄存器时，暂停 PC/IF_ID 一拍，并向 ID/EX 插入 bubble。本实验报告中要说明这个原理，同时说明当前代码主要通过 MEM/WB 转发覆盖验收程序中的数据相关。

控制冒险：分支、jal、jalr 会改变 PC。流水线在 EX 阶段确定跳转后，前面可能已经取入错误路径指令，所以代码用 jump_flush 清空 IF/ID 和 ID/EX 的输入，相当于插入 NOP。

```
assign forward_RD1 = MEM/WB 命中 rs1 ? 转发值 : EX_RD1
assign forward_RD2 = MEM/WB 命中 rs2 ? 转发值 : EX_RD2
assign jump_flush = (EX_NPCOp != NPC_PLUS4)
IF_ID_in = jump_flush ? 0 : IF_ID_raw_in
ID_EX_in = jump_flush ? 0 : ID_EX_raw_in
```

3.5 其他模块与测试程序

im 指令存储器：保存 32 位机器码。PC 是字节地址，而指令按 word 存储，所以常用 PC[31:2] 作为 im 地址。testbench 用 readmemh 把 dat 文件装入 im。

dm 数据存储器：服务 lw/sw。sw 时 CPU 给地址和写数据，MemWrite=1 后写入 dmem[addr[8:2]]；lw 时 dm 输出数据，WB 阶段写回 rd。

RF 寄存器堆：就是 x0 到 x31。它有两个读端口 rs1、rs2 和一个写端口 rd；RegWrite=1 且 rd 不为 x0 时，在时钟沿写入 WD。x0 永远保持 0。

ctrl 译码模块：根据 opcode、funct3、funct7 判断具体指令，并产生 RegWrite、MemWrite、EXTOp、ALUOp、NPCOp、ALUSrc、WDSel 等控制信号。可以把它理解成 CPU 的指挥模块。

Test_30_Instr.dat：主要检查单条或一组扩展指令，包括算术逻辑、比较、移位、lw/sw、分支、jal/jalr。

学号排序程序：更像完整程序测试，会反复使用循环、分支、子过程调用和返回、连续数据相关以及内存读写。程序把原始学号写到 mem[0x180]，把排序结果写到 mem[0x184]。

学号初始化：程序把学号数字按 4 bit BCD 风格拼成 0x54873530，并写入 mem[0x180]，也就是 dmem[96]。

排序结果：程序把 54873530 看成 5、4、8、7、3、5、3、0 这 8 个数字，经过循环比较和 swap 后得到升序结果 03345578，写入 mem[0x184]，也就是 dmem[97]，同时 x15=03345578。

四、测试结果与截图

四组测试关键结果：

测试 1，单周期 Test_30_Instr.dat: x1=000000cc, x4=00000001, x27=3dcc0000, m0=0000000c, m4=0000055c。

测试 2，流水线 Test_30_Instr.dat: 关键寄存器和内存值与单周期一致，说明流水线新增指令和转发/跳转处理满足该测试。

测试 3，单周期学号排序: x15=03345578, m96=54873530, m97=03345578。

测试 4，流水线学号排序: x15=03345578, 写入 0x180/0x184, m96=54873530, m97=03345578。

图 2 测试 1: 单周期 CPU 跑 Test_30_Instr.dat 的寄存器截图

```
... range parameters $readmemh( ... , mem, $start, $stop); // defaulting to 1504 2000 behav
WARNING: ..\tb\sc_final_tb.v:24: $readmemh(E:\computerlab\ComputerDesignLabs-main\CODExp
127].
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=000000cc
[REG] x2=00000000
[REG] x3=00000000
[REG] x4=00000001
[REG] x5=98763dcc
[REG] x6=98765000
[REG] x7=00001236
[REG] x8=98764c00
[REG] x9=0000000c
[REG] x10=00000561
[REG] x11=00000000
[REG] x12=00000000
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=00000000
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000500
[REG] x19=98763dcc
[REG] x20=00000001
[REG] x21=98765001
[REG] x22=98766236
[REG] x23=00001236
[REG] x24=00000000
[REG] x25=98767236
[REG] x26=00000236
[REG] x27=3dcc0000
[REG] x28=098763dc
[REG] x29=f98763dc
[REG] x30=00000000
[REG] x31=00000000
```

图 3 测试 1: 单周期 CPU 跑 Test_30_Instr.dat 的内存截图

```
[SNAPSHOT] memory
[MEM] m0=0000000c
[MEM] m1=98765001
[MEM] m2=00001236
[MEM] m3=00000561
[MEM] m4=0000055c
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
```

图 4 测试 2: 流水线 CPU 跑 Test_30_Instr.dat 的寄存器截图

```

WARNING: ..\tb\pl_final_tb.v:22: $readmemh(E:\computerlab\ComputerDesignLabs-main\CODE
memaddr = 00000000, memdata = 98763dcc
memaddr = 00000004, memdata = 98765001
memaddr = 00000008, memdata = 00001236
memaddr = 0000000c, memdata = 00000000
memaddr = 00000010, memdata = 0000000c
memaddr = 00000014, memdata = 0000055c
memaddr = 00000018, memdata = 00000561
memaddr = 0000001c, memdata = 00000571
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=000000cc
[REG] x2=00000000
[REG] x3=00000000
[REG] x4=00000001
[REG] x5=98763dcc
[REG] x6=98765000
[REG] x7=00001236
[REG] x8=98764c00
[REG] x9=0000000c
[REG] x10=00000561
[REG] x11=00000000
[REG] x12=00000000
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=00000000
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000500
[REG] x19=98763dcc
[REG] x20=00000001
[REG] x21=98765001
[REG] x22=98766236
[REG] x23=00001236
[REG] x24=00000000
[REG] x25=98767236
[REG] x26=00000236
[REG] x27=3dcc0000
[REG] x28=098763dc
[REG] x29=f98763dc
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=0000000c
[MEM] m1=98765001
[MEM] m2=00001236
[MEM] m3=00000561
[MEM] m4=0000055c
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000

```

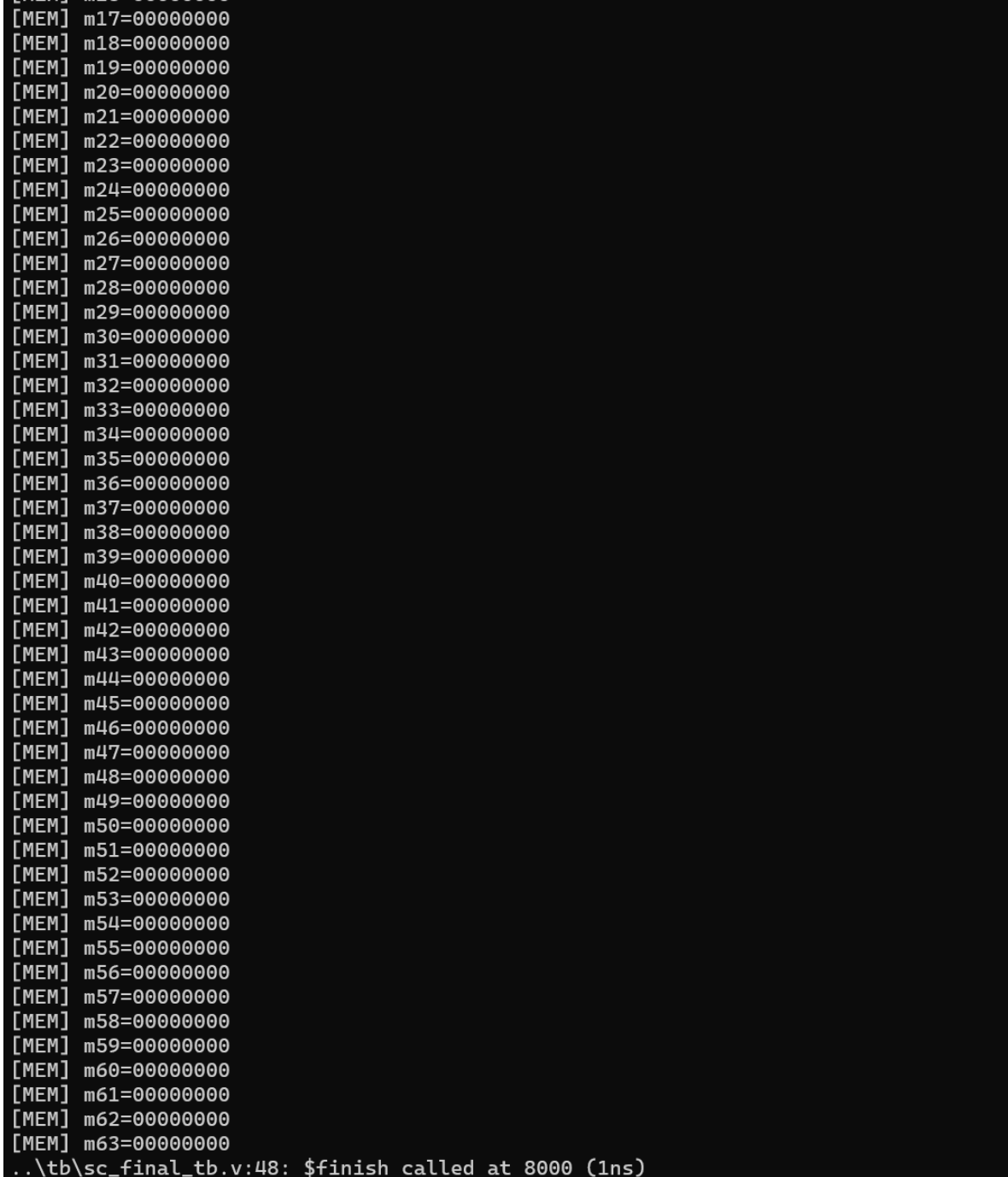
图 5 测试 2: 流水线 CPU 跑 Test_30_Instr.dat 的内存截图

```
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
[MEM] m48=00000000
[MEM] m49=00000000
[MEM] m50=00000000
[MEM] m51=00000000
[MEM] m52=00000000
[MEM] m53=00000000
[MEM] m54=00000000
[MEM] m55=00000000
[MEM] m56=00000000
[MEM] m57=00000000
[MEM] m58=00000000
[MEM] m59=00000000
[MEM] m60=00000000
[MEM] m61=00000000
[MEM] m62=00000000
[MEM] m63=00000000
..\tb\pl_final_tb.v:45: $finish called at 725000 (1ps)
```

图 6 测试 3: 单周期 CPU 跑学号排序程序的寄存器截图, x15 为排序结果

```
.
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=ffff0004
[REG] x2=ffff000c
[REG] x3=00000008
[REG] x4=00000000
[REG] x5=00000100
[REG] x6=00ffffff
[REG] x7=00000000
[REG] x8=03000000
[REG] x9=0000001c
[REG] x10=0000001c
[REG] x11=00000008
[REG] x12=00000007
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=03345578
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000000
[REG] x19=00000000
[REG] x20=00000000
[REG] x21=00000000
[REG] x22=00000000
[REG] x23=00000000
[REG] x24=00000000
[REG] x25=00000000
[REG] x26=00000000
[REG] x27=00000000
[REG] x28=00000000
[REG] x29=00000000
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=00000000
[MEM] m1=00000000
[MEM] m2=00000000
[MEM] m3=00000000
[MEM] m4=00000000
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
```

图 7 测试 3：单周期 CPU 跑学号排序程序的内存窗口截图



```
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
[MEM] m48=00000000
[MEM] m49=00000000
[MEM] m50=00000000
[MEM] m51=00000000
[MEM] m52=00000000
[MEM] m53=00000000
[MEM] m54=00000000
[MEM] m55=00000000
[MEM] m56=00000000
[MEM] m57=00000000
[MEM] m58=00000000
[MEM] m59=00000000
[MEM] m60=00000000
[MEM] m61=00000000
[MEM] m62=00000000
[MEM] m63=00000000
..\tb\sc_final_tb.v:48: $finish called at 8000 (1ns)
```

图 8 测试 4: 流水线 CPU 跑学号排序程序的写内存与寄存器截图

```
WARNING: ..\tb\pl_final_tb.v:22: $readmemh(E:\computerlab\ComputerDesignl  

memaddr = 00000180, memdata = 54873530  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

memaddr = 00000184, memdata = 03345578  

[SNAPSHOT] registers  

[REG] x0=00000000  

[REG] x1=ffff0004  

[REG] x2=ffff000c  

[REG] x3=00000008  

[REG] x4=00000000  

[REG] x5=00000100  

[REG] x6=00ffffff  

[REG] x7=00000000  

[REG] x8=03000000  

[REG] x9=0000001c  

[REG] x10=0000001c  

[REG] x11=00000008  

[REG] x12=00000007  

[REG] x13=00000000  

[REG] x14=00000000  

[REG] x15=03345578  

[REG] x16=00000000  

[REG] x17=00000000
```

图 9 测试 4：流水线 CPU 跑学号排序程序的内存窗口截图（上半部分）

```
[REG] x18=00000000
[REG] x19=00000000
[REG] x20=00000000
[REG] x21=00000000
[REG] x22=00000000
[REG] x23=00000000
[REG] x24=00000000
[REG] x25=00000000
[REG] x26=00000000
[REG] x27=00000000
[REG] x28=00000000
[REG] x29=00000000
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=00000000
[MEM] m1=00000000
[MEM] m2=00000000
[MEM] m3=00000000
[MEM] m4=00000000
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
```

图 10 测试 4：流水线 CPU 跑学号排序程序的内存窗口截图（下半部分）



```
MEM] m29=00000000
MEM] m30=00000000
MEM] m31=00000000
MEM] m32=00000000
MEM] m33=00000000
MEM] m34=00000000
MEM] m35=00000000
MEM] m36=00000000
MEM] m37=00000000
MEM] m38=00000000
MEM] m39=00000000
MEM] m40=00000000
MEM] m41=00000000
MEM] m42=00000000
MEM] m43=00000000
MEM] m44=00000000
MEM] m45=00000000
MEM] m46=00000000
MEM] m47=00000000
MEM] m48=00000000
MEM] m49=00000000
MEM] m50=00000000
MEM] m51=00000000
MEM] m52=00000000
MEM] m53=00000000
MEM] m54=00000000
MEM] m55=00000000
MEM] m56=00000000
MEM] m57=00000000
MEM] m58=00000000
MEM] m59=00000000
MEM] m60=00000000
MEM] m61=00000000
MEM] m62=00000000
MEM] m63=00000000
.\tb\pl_final_tb.v:45: $finish called at 9045000 (1ps)
```

五、开发板运行结果

开发板照片用于证明程序可以在 Nexys A7/Nexys4 DDR 板上完成七段数码管显示。照片中可见数码管显示原始学号与排序/演示结果，说明显示通路、时钟复位和板级约束能够正常工作。

图 11 开发板照片：原始学号显示与排序/演示结果显示

六、代码修改与 diff 说明

本次提交的关键修改集中在单周期和流水线 CPU 的译码、ALU、立即数扩展、下一条 PC 计算、流水寄存器传递和冒险处理。完整源码后续放入 GitHub 公开仓库，报告只展示主要 diff 截图和修改摘要。

主要代码改动：

single-cycle/NPC.v: 新增 RS1 输入和 NPC_JALR 目标计算，支持 jalr 使用 rs1+imm 跳转并清除最低位。

single-cycle/SCCPU.v: NPC 实例连接 .RS1(RD1)，把 RF 读出的 rs1 数据送给 NPC。

single-cycle/alu.v: 扩展 bne/blt/bge/bltu/bgeu、slt/sltu 和移位等 ALUOp。

single-cycle/ctrl.v: 改为 always @(*) + case 译码，更清晰地按 opcode/funct3/funct7 产生控制信号。

pipeline/EXT.v / NPC.v: 增加 B/J 立即数和 JumpPC/RS1，支持流水线分支、jal、jalr 目标地址。

pipeline/PLCPU.v: 增加数据转发、jump_flush 和流水寄存器中文注释，解决 RAW 数据相关和控制冒险。

图 12 diff 截图: single-cycle/NPC.v 增加 RS1 和 NPC_JALR

```

modified single-cycle/NPC.v                                     +3 -1
1      1      `include "ctrl_encode_def.v"
2      2
3      3      - module NPC( PC, NPCOp, IMM, NPC ); // next pc module
4      4      + module NPC( PC, NPCOp, IMM, RS1, NPC ); // next pc module
5      5      input  [31:0] PC;           // pc
6      6      input  [2:0] NPCOp;        // next pc operation
7      7      input  [31:0] IMM;        // immediate
8      8      + input  [31:0] RS1;       // rs1 data for jalr
9      9      output reg [31:0] NPC;    // next pc
10     10     wire [31:0] PCPLUS4;

...

14     15     `NPC_PLUS4: NPC = PCPLUS4; // NPC computes addr
15     16     `NPC_BRANCH: NPC = PC+IMM; //B type, NPC computes addr
16     17     `NPC_JUMP: NPC = PC+IMM;  //J type, NPC computes addr
17     18     + `NPC_JALR: NPC = (RS1 + IMM) & 32'hffff_fffe;
18     19     default: NPC = PCPLUS4;
19     20     endcase
20     21     end
21     22
...

```

图 13 diff 截图: PC/RF 删除调试 \$write 输出

```

modified single-cycle/PC.v                                     +0 -1
...
9      9      PC <= 32'h0000_0000;
10     10     else
11     11     PC <= NPC;
12     12     - $write("\n pc = %h: ", PC);
13     13     end
14     14     endmodule
15     15

...

modified single-cycle/RF.v                                     +0 -1
...
21     21     if (RFwr) begin
22     22     if (A3 != 5'b0) begin
23     23     rf[A3] <= WD;
24     24     - $write("x%d = %h ", A3, WD);
25     25     end
26     26     end
27     27     end
...

```

图 14 diff 截图: SCCPU 中 NPC 连接 RS1(RD1)

```

modified single-cycle/SCCPU.v                                   +2 -2
...
71     71     );
72     72     // instantiation of pc unit
73     73     PC U_PC(.clk(clk), .rst(reset), .NPC(NPC),
74     74     .PC(PC_out));
75     75     - NPC U_NPC(.PC(PC_out), .NPCOp(NPCOp),
76     76     .IMM(immout), .NPC(NPC));
77     77     + NPC U_NPC(.PC(PC_out), .NPCOp(NPCOp),
78     78     .IMM(immout), .RS1(RD1), .NPC(NPC));
79     79     EXT U_EXT(
80     80     .imm(imm), .simm(simm), .bimm(bimm),
81     81     .uimm(uimm), .jimm(jimm),
82     82     .EXTOp(EXTOp), .immout(immout)
83     83
...
89     89     endcase
90     90     end
91     91
92     92     - endmodule
93     93     + endmodule

```

GitHub 公开仓库: <https://github.com/yuxuanhuang641-crypto/computer-design-practice-final-2024302181173>

七、总结

本次最终验收完成了单周期 CPU 与五级流水线 CPU 的扩展指令实现。单周期部分重点是译码、ALU 和 NPC 的组合逻辑补全；流水线部分除了复用这些功能，还额外处理了控制信号跨阶段保存、RAW 数据转发和跳转 flush。四组老师指定测试和自动评分均达到预期，上板照片也完成了数码管显示验证。

答辩时如果老师要求现场指出代码位置，可以优先打开：ctrl.v 看 opcode/funct3/funct7 译码，alu.v 看 add/slt/bne 的运算，NPC.v 看 jalr 的 PC 目标，PLCPU.v 看 forwarding、IF/ID、ID/EX、EX/MEM、MEM/WB 和 jump_flush。